

State management

COMPLETE LEARNING GUIDE

What You'll Learn

This submodule focuses specifically on React's state management toolkit, from the simplest case (a single component's own `useState`) through sharing state across a subtree with `Context`, up to reaching for a dedicated library like `Redux` (or its modern, far less boilerplate-heavy successor, `Redux Toolkit`) once an application's shared state grows complex enough to need more structure. The core skill isn't memorizing any one API — it's judgment: recognizing when state genuinely needs to live in a component versus be lifted to a parent, versus be shared via `Context`, versus warrant a dedicated state library, and understanding the real trade-offs (re-render behavior, boilerplate, debuggability) each option carries.

1. `useState`: Local Component State

`useState(initialValue)` returns a `[value, setValue]` pair — reading the current value and a setter that schedules a re-render with the new value. React batches multiple `setState` calls that happen within the same event handler into a single re-render for performance, and state updates are asynchronous relative to the code that triggered them — reading the state variable immediately after calling its setter still returns the old value, since the update hasn't applied yet. When a new state value depends on the previous one, passing a function to the setter (`setCount(prev => prev + 1)`) guarantees you're operating on the latest value, which matters especially inside closures like event handlers or timers.

JSX

```
function Counter() {
  const [count, setCount] = useState(0);

  function handleTripleIncrement() {
    // WRONG: all three reference the same stale `count` from this render
    // setCount(count + 1); setCount(count + 1); setCount(count + 1);

    // RIGHT: functional updates always see the latest value
    setCount((prev) => prev + 1);
    setCount((prev) => prev + 1);
    setCount((prev) => prev + 1); // count increases by 3
  }

  return <button onClick={handleTripleIncrement}>{count}</button>;
}
```

2. Lifting State Up

When two sibling components need to share or stay in sync on the same piece of state, the state should live in their closest common ancestor, which then passes the value down as props to both and passes down a callback that lets either child request a change. This is 'lifting state up' — React has no direct sibling-to-sibling communication, so a shared parent acting as the single source of truth is the standard solution before reaching for Context or an external store.

JSX

```
function FilterableList() {
  const [searchTerm, setSearchTerm] = useState('');
  // Both siblings share `searchTerm`, lifted to their common parent
  return (
    <>
      <SearchBox value={searchTerm} onChange={setSearchTerm} />
      <ResultsList searchTerm={searchTerm} />
    </>
  );
}
```

3. Context API: Avoiding Deep Prop Drilling

Prop drilling — passing a prop down through several layers of components that don't use it themselves, just to reach a deeply nested consumer — becomes unwieldy past a couple of levels. `createContext()` plus a `Provider` lets any descendant read a value directly via `useContext()`, without every intermediate component needing to know or forward it. Context is best suited to genuinely cross-cutting values that many distant components need (current theme, authenticated user, locale) — it is not a general-purpose replacement for all shared state, since every consumer of a context re-renders whenever the context's value changes, even if the consumer only cares about part of that value.

JSX

```
const ThemeContext = createContext('light');

function App() {
  const [theme, setTheme] = useState('light');
  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <Dashboard /> {/* any descendant, at any depth, can now read the theme */}
    </ThemeContext.Provider>
  );
}

function ThemeToggleButton() {
  const { theme, setTheme } = useContext(ThemeContext); // no prop drilling
  return (
    <button onClick={() => setTheme(theme === 'light' ? 'dark' : 'light')}>
      Current: {theme}
    </button>
  );
}
```

4. useReducer for Complex Local State

When a single component's state involves multiple sub-values that change together in response to distinct 'actions' (e.g. a form with loading/error/data states, or a shopping cart with add/remove/clear operations), useReducer often produces clearer code than several separate useState calls updated ad hoc across many handlers. A reducer is a pure function (state, action) => newState, and the component dispatches plain action objects describing what happened rather than directly computing the next state inline in each handler — the same pattern Redux is built entirely around, just scoped to one component.

JSX

```
function cartReducer(state, action) {
  switch (action.type) {
    case 'add': return [...state, action.item];
    case 'remove': return state.filter((i) => i.id !== action.id);
    case 'clear': return [];
    default: return state;
  }
}

function Cart() {
  const [items, dispatch] = useReducer(cartReducer, []);
  return (
    <button onClick={() => dispatch({ type: 'add', item: { id: 1, name:
'Widget' })}}>Add to cart ({items.length})</button>
  );
}
```

5. Redux Basics: Store, Actions, Reducers

Redux applies the same reducer pattern globally, at the whole-application level: a single store holds the entire app's shared state, components dispatch plain action objects describing intent ({ type: 'cart/add', payload }), and one root reducer computes the next state from the current state and the dispatched action, always immutably (never mutating the existing state object directly). Modern Redux code is almost always written with Redux Toolkit, which drastically cuts the boilerplate of hand-written Redux by generating action creators and reducers together via createSlice, and by allowing 'mutating' syntax inside reducers that's safely converted to immutable updates under the hood (via the Immer library).

JAVASCRIPT

```
import { createSlice, configureStore } from '@reduxjs/toolkit';

const cartSlice = createSlice({
  name: 'cart',
  initialState: { items: [] },
  reducers: {
    addItem: (state, action) => {
      state.items.push(action.payload); // looks mutating, Immer makes it safe
    },
    clearCart: (state) => { state.items = []; },
  },
});

export const { addItem, clearCart } = cartSlice.actions;
export const store = configureStore({ reducer: { cart: cartSlice.reducer } });
```

6. Choosing Local State, Context, or a Store

A practical decision path: default to useState/useReducer scoped to the component that needs it; lift state to the nearest shared ancestor only once a sibling genuinely needs it too; reach for Context once state needs to be read by many components at very different depths and re-render cost of the whole subtree is acceptable; reach for Redux (or a lighter alternative like Zustand) once an application has enough interconnected shared state that tracking who-changes-what informally becomes error-prone, or once you need Redux DevTools' time-travel debugging and middleware ecosystem. Most real applications end up using all three at different points, not just one exclusively.

7. Avoiding Unnecessary Re-Renders from Context

Because every consumer of a Context re-renders whenever its value changes, a Context whose value is a large object holding many unrelated pieces of state causes far more re-rendering than necessary — a component that only reads one field still re-renders when any other field changes. Splitting one large Context into several smaller, more targeted Contexts (one for theme, one for auth, one for cart) confines re-renders to only the consumers that actually care about that particular slice of state, which matters increasingly as an app's component tree grows.

Common Pitfalls

- Reading a state variable immediately after calling its setter and expecting the updated value — state updates are asynchronous relative to that line of code.
- Calling a setter multiple times based on the current (stale) value instead of using the functional update form (`prev => ...`), losing updates.

- Mutating a state object or array directly (`state.items.push(x)`) instead of creating a new reference, which prevents React from detecting the change.
- Putting a single giant object with many unrelated fields into one Context, causing every consumer to re-render on any change to any field.
- Reaching for Redux immediately for a small app's simple, localized state instead of starting with `useState/useReducer`.
- Forgetting that Context re-renders every consumer on value change, and using it for high-frequency updates (like live cursor position) where performance suffers.

Key Takeaways

- `useState`'s setter is asynchronous relative to surrounding code — use the functional update form when the new value depends on the previous one.
- Lift state to the nearest common ancestor before reaching for Context; reach for Context before reaching for a full external store.
- Never mutate state directly — always create a new object/array reference so React can detect the change and re-render.
- Redux Toolkit's `createSlice` removes almost all of classic Redux's boilerplate while keeping the same store/action/reducer model.
- Splitting a large Context into several smaller ones limits unnecessary re-renders to only the components that actually depend on each slice.

Learn More

React — Managing State

<https://react.dev/learn/managing-state>

React — Passing Data Deeply with Context

<https://react.dev/learn/passing-data-deeply-with-context>

Redux Toolkit — Official Documentation

<https://redux-toolkit.js.org/>